▸ **Aufgabe 1: Extended Loop ⇒ Sprache** [4+4 Punkte]

Sei Σ = {A, B, C} das Alphabet. Definieren Sie zu jedem der beiden folgenden Extended Loop-Programme die Sprache L ⊆ Σ*, die dieses Programm akzeptiert! Verwenden Sie dabei die Notation "L = {...}"!

Ich werde in meiner Lösung so vorgehen: Ich werde erstmal das Programm erklären und anhand der Erklärung werden wir die Sprache definiert ist

1.
```
n ← len(in)
if n % 2
   return false
end
h ← n / 2
i ← 0
j ← h
loop h
   if in[i] ≠ in[j]
      return false
   end
   ++i
   ++j
end
return true
```

in bekommt die Länge vom Input (String)

prüft ob input durch 2 teilbar ist, also ob in gerade ist, falls input nicht gerade ist wird false zurückgegeben, falls in = gerade wird das Programm weiter bearbeitet.

h ist ein index wird positioniert index Mitte

i " " wird Position 0 am Anfang des Strings

wird genau wie h positioniert

→ Hier haben wir eine Schleife, läuft h-mal durch und in jeder Schleifendurch wird geprüft ob das Zeichen im Position i ≠ Zeichen im Position j

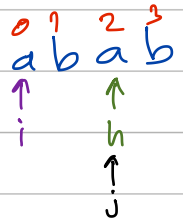
Cata ergibt false, a≠b ergibt true),

Also das Zeichen im Position i und Position j sollen gleich sein, wenn die gleich sind wird i und j erhöht

Wenn die Zeichen ≠ sind, dann wird false zurück gegeben und die Abarbeitung des Programms wird beendet

Beispiel: in = ababa n = 5, 5 % 2 = 1 → false

in = abab n = 4, 4 % 2 = 2 → Programm wird abgearbeitet



h = 4/2 = 2

j ← h

i ← 0

loop h (=2)

if in[i] ≠ in[j]

return false

end

++i

++j

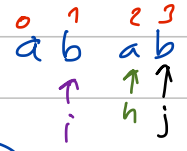
end

1. Iteration

a ≠ a (false)

++i (i = 1)

++j (j = 3

⇒ 

2. Iteration

b ≠ b (false)

++i (i = 2)

++j (j = 4)

⇒ Abarbeitung endet hier da nur 2-Durchläufe

Erkenntnisse: Sprache soll gerade und

Das programm akzeptiert alle Wörter, die sich in zwei gleiche Hälften zerlegen lassen,

"$L = \{ ww \mid w \in \{A,B,C\}^* \}$"

z.B. A|A, AB|AB, CA|CA, ABC|ABC $\in L$

ABA, ABC $\notin L$ weil ungerade

ABBB $\notin L$ weil ist gerade ab erste Hälfte ≠ zweite Hälfte von string.

Analyse

```
2.  n ← len(in)
    i ← 0
    ca ← 0
    cb ← 0
    for i ← 0, ..., n−1
      if in[i] = "A"
        ++ca
      end
      if in[i] = "B"
        ++cb
      end
    end
    if ca = cb
      return true
    else
      return false
    end
```

n = Länge vom String (input) in

Counter i für die Schleife

Counter für "A"

Counter for "B"

Das ist eine for schleife es sind ((n-1) - 0) + 1 = n

Durch läufe vom ersten zeichen bis zum letzten zeichen

wenn ein "A" im String gefunden wird, wird der zähler von A erhöht

wenn ein B gefunden wird, wird der zähler von B inkrementiert

wenn Anzahl von A's = Anzahl von B's gleich sind, dann gibt das Programm true zurück

sonst gibt das programm false zurück.

"$L = \{ w \in \{A,B,C\}^* \mid \text{Anzahl von A in } w = \text{Anzahl von B in } w \}$"

akzeptierte strings: AABBC, AABB, BBAA, BBAACC

BCC, CCBB werden abgelehnt.

Augabe 2 :
L = {A^nBC^n! | n from N }

paar Anmerkungen für dieses Programm:
 wenn n=1 : der String = ABC (das wäre hier der minimum string, der die Sprache akzeptiert)
 wenn n=2 : der String = AABCC
 wenn n=3: der String = AAABCCCCCC
 wenn n=4 der String =AAAABC.....C (4 A, 1 B, 24 C)
 mit n=1 ist die Länge vom String =3
 mit n=2 ist die Länge vom String =5
 mit n=3 ist die Länge vom Stirng =10
 mit n=4 ist die Länge vom String = 29
 1. Beobachtung länge ist nicht eindeutig für mich einmal gerade einmal nicht nicht gerade
 2. Beobachtung Anzahl von A soll gleich n sein gefolgt von geneu einem B dann folgt genau anzahl von C = n! womit A kommt vor B und B kommt vor C
 3. Mindest Länge =3 D.h. jeder String der 1 oder 2 zeichen besteht wird abgelehnt
 4. kommt ein B vor A oder kommt ein C vor B ==> wird der String abgelehnt

 Es gibt eine beliebige Anzahl n von A's zu Beginn.
 Dann genau ein B.
 Dann genau ein n! von C.

 Lösung:

```
 n ← len (in)
if n < 3
    return false  // Mindestlänge ist 3 (ABC)
end

// Initialisierung der Zähler und Flags
i ← 0
count_A ← 0
count_B ← 0
count_C ← 0
foundB ← false
foundC ← false

// Zähle die A's und finde n
loop n
    c ← in[i]

    if c = code(A)
        if foundB or foundC
            return false  // B oder C nach A
        end
        ++count_A  // Zähle die A's
    end

    if c = code(B)
        if foundB or foundC
            return false  // Mehr als ein B oder C vor B
        end
        foundB ← true
        ++count_B  // Zähle das B
    end

    if c = code(C)
        if not foundB
            return false  // C vor B ist nicht erlaubt
```

```
        end
        foundC ← true
        ++count_C  // Zähle die C's
    end
end

// Fakultätsberechnung von n
factorial ← 1
temp ← count_A  // temp = n
loop temp
    factorial ← factorial * temp
    temp ← temp - 1
end

// Prüfung der Bedingungen
if count_A > 0 and count_B = 1 and count_C = factorial
    return true
else
    return false
end
```

Erklärung:

1.Längenprüfung: Das Programm gibt false zurück, wenn die Eingabe kürzer als 3 Zeichen ist.

2.Zähler: Wir zählen A's, B und C's separat.

3. Reihenfolgeprüfung:
 B darf nur einmal vorkommen.
 C darf nur nach B kommen.

4.Fakultätsberechnung: Die Fakultät von n (also die Anzahl der A's) wird in einer separaten Schleife berechnet.
5.Validierung: Es wird geprüft, ob genau n A's, 1 B und n! C's vorliegen.

```
// Aufgabe 3 Biite mit Notepad öffnen

n ← len(in)
i ← n−1
while in[i] ≠ "B"
      if in[i] = "A"
            return true
      end //zu if
      −−i
end // gehört zu while
return false
```

Analyse vom Programm :

n ist die Laenge vom String (input) = in
i ist der Index am letzten Position (also i zeigt auf das letzte Zeichen)
dann gibt eine Schleife prüft ob das Ziechen an der stelle i ≠ "B" falls das der fall ist, wird die Schleife bearbaietet sonst gibt das Programm false zurück
in der Schleife gibt es eine if Bedingung, sie prüft ob das zeichen an der stelle i = "A" ist das der Fall gibt das Programm ture zurück, wenn nicht wird i dekrementiert bis "A" gefunden wird
ganz wichtig zu beachten : jedes Mal wo i dekrementiert wird, wird die Schelifenbedingung geprüft

Beispiele für Schelifenbedingung:
("B" ≠ "B" also das letzte Zeichen ist "B" die Auswertung ergibt false)
Schleife wird nicht abgearbeitet und gibt ein false zurück
("A" ≠ "B" , das letzte Zeichen ist "A" die Auswertung ergibt true) Schleife wird abgearbeit
("C" ≠ "B", das letzte Zeichen ist "C" die Auswertung ergibt true) Schleife wird abgearbeitet

was akzeptiert das Programm also wann ergibt er true ?

Es akzeptiert alle Strings die aus dem Alphabet Σ = {A, B, C} mit der Eigenschaft der String enthält mind. ein "A" bevor ein "B" gefunden wird.

1. Geben Sie ein Beispiel für einen Eingabestring, den das Programm akzepiert, d.h. für den das
Programm den Wert true zurückgibt!
2. Geben Sie ein Beispiel für einen Eingabestring, den das Programm ablehnt, d.h. für den das Programm
den Wert false zurückgibt!
3. Geben Sie ein Beispiel für einen Eingabestring, bei dem sich das Programm "aufängt", d.h. für den die
durch das Programm berechnete Funkon nicht definieert ist!
4. Schreiben Sie ein Programm in der Programmiersprache Extended Loop, das dieselbe Sprache
akzepert, wie das oben aufgeführte Extended-While-Programm! Beide Programme sollen also für
genau dieselben Eingabestrings den Wert true zurückgeben.

Antwort zu

1. ist die Eingabe : ABCBA (es wird direkt von hinten ein A gefunden und das Programm gibt true zurück)
2. die Eingabe ABAB (das Programm trifft direkt auf ein B , Schleife wird nicht abgearbeit und gibt false zurück)
3. das Programm kann nicht in eine endloseschleife
 gehen weil der counter i läuft von hinten nach vorn im string und wird bei

jeder iteration dekrementiert und daher kann es nicht sein dass die while schleife endlos wird

4.

```
n ← len (in)
i ← n-1
loop i
      if in [i] = "B"
            return false
      end

      if in[i] = "A"
            return true
      end
      --i
end
return false
```

Analyse :
n ist die Länge vom input
i ist der letzte index
schleife läuft n-mal durch den string
wenn ein B gefunden wird gibt die Schleife false zurück
wenn mind. ein A gefunden wird gibt die Schleife true zurück und in jedem Durchlauf wird i dekrementiert.

Wir erinnen uns Jedes loop Programm ==> While Programm umgewandelt werden
aber nicht umgekehrt
aber in diesem Fall lässt sich umwandeln weil das Programm nicht endlos laeuft .